Chapter 7: Flow-control Constructs

Mohamed M. Sabry Aly
N4-02c-92

1

Learning Objectives

- Be able to convert a high level IF and IF-ELSE construct to its low-level equivalent.
- Describe compute-efficient considerations for compound AND and OR constructs.
- Describe and analyze simple branchless logic constructs.

2

IF Statements

- How is the **IF** construct implemented?
- **Imitating** the high-level test condition does not result in very efficient assembly-level implementation.

```
if (a > b)
  {S1}
```

Convert to assembly code

```
      CMP a,b
      BGT DoIF
      B Skip
DoIF  {S1}
Skip
```

- High-level test condition is **reversed** in assembly-level to avoid the need for an additional unconditional jump.

```
if (a > b)
  {S1}
```

Convert to assembly code

```
      CMP a,b
      BLE Skip
      {S1}
Skip
```

Note: The reverse condition test of **HS** is **LO**, reverse of **LT** is **GE**

3

Find Largest Number

```
      MOV  R0,#0x100    ;setup pointer to first array element
      MOV  R1,#9
      LDR  R3,[R0]
Loop  LDR  R2,[R0,#4]!
      CMP  R2,R3        ;compare R3 and R2 (i.e. R2-R3)
      BLO  Skip         ;branch if R2 <  current max (i.e. R3)
      MOV  R3,R2        ;update current max. in R3 with R2
Skip  SUBS R1,R1,#1     ;decrement 1 from counter register
      BNE  Loop         ;jump back to Loop if not zero
```

```
if R2 >= R3   // R2 larger or equal to current max
{
  R3 = R2;    // then update R3 with new max R2
}
```

R0 = Address pointer for current array element.
R1 = Loop counter register
R2 = Temporary register holding current no.
R3 = Current maximum value (i.e. the result).

4

Can We Avoid Reversion?

```
      MOV   R0,#0x100     ;setup pointer to first array element
      MOV   R1,#9
      LDR   R3,[R0]
Loop  LDR   R2,[R0,#4]!
      CMP   R2,R3
      BLO   Skip
      MOV   R3,R2
Skip  SUBS  R1,R1,#1
      JNE   Loop
```

```
if R2 >= R3   // R2 larger or equal to current max
{
  R3 = R2;    // then update R3 with new max R2
}
```

```
      CMP   R2,R3
      BHS   Assign
      SUBS  R1,R1,#1
      BNE   Loop
Assign MOV  R3,R2
      SUBS  R1,R1,#1
      BNE   Loop
```

Redundant

Conditional Execution

- In the 32-bit ARM ISA, instructions can be conditionally executed based on the CC flags.
- Consider the following 32-bit ARM code segment.

```
; C code
if (r0 == 1)
 r1 = 3;
```

```
      CMP   r0, #1        ; set CC based on r0 –1
      BNE   Skip          ; if (r0 == 1)
      MOV   r1, #3        ; then { r1 := 3}
SKIP    .....             ;
```

- It can be replaced using conditional execution instructions.

```
      CMP   r0, #1        ; if (r0 == 1)
      MOVEQ r1, #3        ; then { r1 := 3}
SKIP  ....._              ;
```


Largest with Conditional Execution

- Significant reduction in Code size

```
      MOV   R0,#0x100     ;setup pointer to first array element
      MOV   R1,#9
      LDR   R3,[R0]
Loop  LDR   R2,[R0,#4]!
      CMP   R2,R3         ;compare R3 and R2 (i.e. R2-R3)
      MOVGE R3,R2         ;update current max. in R3 with R2
      SUBS  R1,R1,#1      ;decrement 1 from counter register
      JNE   Loop          ;jump back to Loop if not zero
```

```
if R2 >= R3   // R2 larger or equal to current max
{
  R3 = R2;    // then update R3 with new max R2
}
```


Does Conditional Execution work with Just 1 Instruction?

No, as long as the status registers are not altered from the comparison instruction

Find Largest Number, and Store Its Index

No conditional execution

```
R3= X[0];
R4=0;
R0= &X[0]
For (R1=9;R1>0;R1--){
      R0+=4;R2=X[R0];
      if(R2>=R3){
            R3=R2;
            R4=10-R1;
      }
}
```

```
      MOV   R0,#0x100     ;setup pointer to first array element
      MOV   R1,#9         ;load 9 into counter register
      MOV   R4,#0         ;load 0 into index_max register
      LDR   R3,[R0]       ; 1st no. in array is current max
Loop  LDR   R2,[R0,#4]!   ;get next no. in array
      CMP   R2,R3         ;compare R3 and R2 (i.e. R2-R3)
      BLO   Skip          ;branch if R2 <  current max (i.e.
      MOV   R3,R2         ;update current max. in R3 with R2
      RSUB  R4,R1,#10     ;Store the index in R4
Skip  SUBS  R1,R1,#1      ;decrement 1 from counter register
      BNE   Loop          ;jump back to Loop if not zero
```


9

Find Largest Number, and Store Its Index

With conditional execution

```
R3= X[0];
R4=0;
R0= &X[0]
For (R1=9;R1>0;R1--){
      R0+=4;R2=X[R0];
      if(R2>=R3){
            R3=R2;
            R4=10-R1;
      }
}
```

```
      MOV   R0,#0x100     ;setup pointer to first array element
      MOV   R1,#9         ;load 9 into counter register
      MOV   R4,#0         ;load 0 into index_max register
      LDR   R3,[R0]       ; 1st no. in array is current max
Loop  LDR   R2,[R0,#4]!   ;get next no. in array
      CMP   R2,R3         ;compare R3 and R2 (i.e. R2-R3)
      MOVGE R3,R2         ;update current max. in R3 with R2
      RSUBGE R4,R1,#10    ;Store the index in R4
      SUBS  R1,R1,#1      ;decrement 1 from counter register
      BNE   Loop          ;jump back to Loop if not zero
```


10

IF-ELSE Statements

- How is the **IF-ELSE** construct implemented?
- Combination of conditional and unconditional jumps used for the **IF-ELSE** construct.
- Reversing high-level test condition does not improve efficiency unless
- the ELSE code segment {S2} is more likely to execute.

```
      CMP a,#3
      BNE DoElse
      {S1}
      B Skip
DoElse {S2}
Skip   :
```

Reversing test condition

```
if (a == 3)
  {S1}
else
  {S2}
```

Convert to assembly code

```
      CMP a,#3
      BEQ DoIf
      {S2}
      B Skip
DoIf  {S1}
Skip   :
```

IF-ELSE implementation in low-level assembly

11

Conditional Execution for IF-ELSE

- Higher efficient coding
- In the shown example, Skip will always be the 4th instruction

```
; C code
if (r0 == 1)
 r1 = 3;
else
 r1 = 5;
```

```
      CMP   r0, #1        ; set CC based on r0 –1
      BNE   ELSE          ; if (r0 == 1)
      MOV   r1, #3        ; then { r1 := 3}
      B     SKIP          ; skip over else code seg
ELSE  MOV   r1, #5        ; else  { r1 := 5}
SKIP    .....             ;
```

- With conditional execution

```
      CMP r0, #1          ; if (r0 == 1)
      MOVEQ r1, #3        ; then { r1 := 3}
      MOVNE r1, #5        ; else  { r1 := 5}
SKIP  .....               ;
```


12

Compound AND Conditions

- How are compound AND conditions handled?
- Logical AND can bind multiple basic relational conditions.

e.g.
```
if ((a==b) && (b>0)) {S1}
```
order of compound AND test

- Compilers resolve compound conditions into simpler ones.

```
    if (a!=b) then Skip
    if (b<=0) then Skip
       {S1}
Skip    :
```

- Elementary conditions bound by the logical AND are tested from **left-to-right**, in the order given in the C program.
- The first false condition means the remaining conditions are not computed. This is called the **fast Boolean operation**.
- Keep the **least likely** condition **leftmost** in your program for more efficient execution.

Compound Condition Example

- Not all cases will be executed correctly

```
if ((a==b) && (b>0)) {S1}
```

What if a>b and b>0

```
CMP     R1,R2 ;R1<-a, R2<-b
CMPEQ   R2,#0
XXXGT   XXXX  ; S1
```


Compound OR Conditions

- How are compound OR conditions handled?

e.g.
```
if ((a==1) || (a==2)) {S1}
```

- Most compilers eliminate an unconditional jump at the end of the compound OR series by **reversing** the **last conditional** test.

```
     if (a==1) then DoIf
     if (a!=2) then Skip
DoIf {S1}
Skip   :
```

- The conditional test that is **most likely** to be **true** should be kept leftmost.

Compound Condition Example

```
if ((a==1) || (a==2)) {S1}
```

With conditional assignment
S1 needs to be replicated.
Not suitable for multiple instructions

```
CMP     R1,#1 ;R1<-a
XXXEQ   XXXX  ; S1
CMPNE   R1,#2
XXXEQ   XXXX  ; S1
```

A more holistic approach

```
     CMP     R1,#1
     BEQ     doIF
     CMPNE   R1,#2
doIF XXXEQ   XXXX  ; S1
```

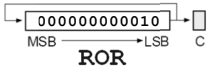
Branchless Logic

- Branchless logic avoid using conditional jump instructions when implementing logical constructs.
- `Bcc` instructions may result in costly **flushing** operations when the wrong next instruction is pre-fetched into the CPU's pipeline.
- How is branchless logic implemented?
- **Exploit arithmetic relationship** to transform the test condition into the corresponding desired outcome. Can only be applied in special cases and desired outcomes are usually Boolean values.

e.g.

```
if ((X & 2) == 2)
   X = true;
else
   X = false;
```

```
AND R1,R1,#0x02
ROR R1,R1,#1
```



- **Conditional execution** can be used to avoid branching.

Summary

- The **IF** and **ELSE** constructs are implemented using one or more conditional jump (`Bcc`).
  - Using the **reverse** conditional test can help the IF construct execute more efficiently.
- **Conditional execution in ARM** greatly improves flow-control efficiency.
- Sequencing the **least likely** or **most likely** conditional test can help improve execution speed of compound **AND** and **OR** respectively .
- **Branchless logic** and **conditional execution** techniques can help keep the CPU pipeline efficient by maintaining **sequential** execution.